

# Week 2 - Lecture 4 - Problem: Contact Book

---

## ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data, depending on their performance characteristics.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs by the use of iterations, functions, modules, classes, and methods.
- **CS3:** Form readable, descriptive and well-documented program code.
- **CS6:** Build basic interactive programs with text-based and graphical user interfaces.
- **CS9:** Use standard libraries and follow best programming practices.

## Objectives

After completing this problem, you should be able to:

1. Store a contact as a single string containing both name and phone number.
2. Store several such strings in one list.
3. Split a contact string to separate the name from the phone number.
4. Recognize the risk of keeping name and phone number combined in one string.
5. Combine name and phone number into a tuple instead.
6. Store several tuples in a single list of contacts.
7. Add a new contact to the list.
8. Search for a contact by name.
9. Delete a contact by name.
10. Build a menu loop that repeats until the user chooses to quit.

## Background

A simple contact book stores a name together with a phone number. A first attempt at this might store each contact as one string, for example "Alice 070-1112233", and keep all these strings in a single list. To use the name or the phone number separately, the string must be split apart every time. This works, but it is fragile: the split only works correctly if the format of every string is exactly the same, and it is easy to make a mistake when reading or building the string.

A better design (which we'll implement later) stores each contact as a tuple (name, phone\_number). All contacts can then be kept in one list of tuples, so the name and phone number stay together as separate values, not glued into one string.

## Task

### Part 1: One combined string per contact

1. Create a list called `contacts` where each element is one string containing a name and a phone number, for example "Alice 070-1112233".
2. For each contact string, split it to get the name and the phone number separately.
3. Print each name together with its phone number.

4. Add a new contact by appending a new combined string to the list.
5. Discuss what could go wrong with this approach (for example, a name with a space in it, or a missing phone number, breaking the split).

## Part 2: List of tuples

1. Combine each name and phone number into a tuple, `(name, phone_number)`.
2. Store all contact tuples in a single list called `contacts`.
3. Print every contact by looping over the list, unpacking each tuple into `name` and `phone_number`.
4. **Add contact:** append a new `(name, phone_number)` tuple to `contacts`.
5. **Search contact:** loop over `contacts`, and print the phone number of the contact whose name matches, or a message if no match is found.
6. **Delete contact:** loop over `contacts` to find the tuple whose name matches, then remove that tuple from the list using `.remove()` or by rebuilding the list without it. Print a message if no matching contact is found.

## Part 3: Menu loop

1. Wrap the add, search, delete, and print operations from Part 2 in a menu that repeats using a `while` loop.
2. Each time through the loop, print the menu options and read the user's choice.
3. Call the matching operation based on the choice:
  - Add contact
  - Search contact
  - Delete contact
  - Print all contacts
  - Quit
4. Use `break` to exit the loop when the user chooses to quit.
5. Use `continue` or an `else` branch to handle an invalid choice, then show the menu again.

## Example Run

```
-- Part 1: combined strings --
Alice: 070-1112233
Bob: 070-4445566
Carol: 070-7778899

-- Part 2: list of tuples --
Alice: 070-1112233
Bob: 070-4445566
Carol: 070-7778899

Add contact: Dana 070-0001122
Alice: 070-1112233
Bob: 070-4445566
Carol: 070-7778899
Dana: 070-0001122

Search for: Bob
Bob's number is 070-4445566
```

```
Delete: Carol
Contact Carol deleted.
Alice: 070-1112233
Bob: 070-4445566
Dana: 070-0001122

-- Part 3: menu loop --
Contact Book
1. Add contact
2. Search contact
3. Delete contact
4. Print all contacts
5. Quit
Choose an option: 1
Enter name: Eve
Enter phone number: 070-3334455
Contact Eve added.

Contact Book
1. Add contact
2. Search contact
3. Delete contact
4. Print all contacts
5. Quit
Choose an option: 5
Goodbye.
```

## Requirements

- Use one list of combined strings in Part 1, and split each string to read the name and phone number.
- Use tuples in Part 2 to keep each name and phone number as separate values.
- Use a list to store all contact tuples.
- Use a loop to add, print, search, and delete contacts.
- Use tuple unpacking when reading each contact, for example `name, phone_number = contact`.
- When deleting, handle the case where the name is not found in `contacts`.
- In Part 3, use a `while` loop that keeps showing the menu until the user quits.
- In Part 3, use `break` to leave the loop and handle an invalid menu choice without crashing.